

SD Dictionaries - Formats and Expressions

Format specifications, l-type expressions, correlatives, and chaining conversions.

This page continues SD Dictionaries - Conversions and Formatting.

Format specifications — field 5

Field 5 controls how the query processor lays out the column. It is also the argument to the BASIC `FMT()` function, and the syntax is the same.

```
{length} {fill.char} justification {decimal_places} {options}
```

PART	MEANING
<i>length</i>	field width in characters
<i>fill.char</i>	a quoted character to pad with (defaults to space)
justification	<code>L</code> , <code>R</code> , <code>T</code> , <code>C</code> or <code>U</code>
<i>decimal_places</i>	number of decimal places (0-9)
<i>scale_factor</i>	a second digit — the scale factor

JUSTIFICATION

CODE	MEANING
<code>L</code>	left-justified
<code>R</code>	right-justified
<code>T</code>	text — left-justified, wrapping at word boundaries
<code>C</code>	centred
<code>U</code>	left-justified, unchanged — no trimming

NUMERIC OPTIONS

After the justification, a format can carry these flags:

FLAG	MEANING
\$	prefix with the currency symbol
,	insert thousands separators
Z	suppress zero — print nothing for zero
B	blank if negative — print spaces for a negative number
C	suffix <code>CR</code> for negative, blank for positive
D	suffix <code>DB</code> for negative, blank for positive
E	enclose negative in angle brackets
M	suffix minus for negative
N	prefix minus for negative

EXAMPLES

FORMAT	INPUT	OUTPUT
10L	hello	hello
10R	123	123
10R2	1234.5	1234.50
10R2,	1234567.89	1,234,567.89
10R2\$	1234.5	\$1234.50
8L#	hello	###hello
20T	long text	wrapped at 20 characters
10RZ	0	(empty)

DATE SHORTCUT — D IN THE FORMAT FIELD

If the format field begins with `D`, it is not a format at all — it is a date conversion. `D2[DD/MM/YY]` in field 5 does the same as putting the same string in field 3. This is a convenience: a D-type record can carry its date conversion in either field.

HOW CONVERSION AND FORMAT INTERACT

Conversion (field 3) and format (field 5) are applied in sequence: the conversion transforms the stored value first, then the format controls how the converted value is laid out in the column.

STEP	WHAT HAPPENS
1. Stored value	the raw data from the record field
2. Conversion (field 3)	transforms the value — date formatting, numeric masking, case conversion, translation lookup
3. Format (field 5)	controls width, justification, padding, and numeric display options

A date field stores an internal date (days since 31 Dec 1967). The conversion `D2[DD/MM/YY]` turns `20899` into `29/08/26`. The format `10L` left-justifies it in a 10-character column. Both are needed: the conversion makes the value readable, the format makes the column neat.

A format cannot convert. If the format field begins with `D`, it is a date conversion, not a format — see above. But a format that begins with a digit, `L`, `R`, `T`, `C` or `U` is always a layout specification and never transforms the data. If you need the data transformed, put the conversion in field 3.

When both fields are present, the conversion runs first and the format is applied to the converted result. When only the format is present, it is applied to the raw stored value. When only the conversion is present, the converted value is printed with a default format — the query processor works out a width from the data.

COMBINED EXAMPLES

Every row below was run on W1.0-0 and shows what came back, including the trailing spaces the format adds.

FIELD 3 (CONVERSION)	FIELD 5 (FORMAT)	STORED VALUE	OUTPUT
<code>D2/</code>	<code>10L</code>	<code>20899</code>	<code>03/20/25</code>
<code>D</code>	<code>12L</code>	<code>20899</code>	<code>20 MAR 2025</code>
<code>MD2</code>	<code>10R2,</code>	<code>1234567</code>	<code>12,345.67</code>
<code>MD2,</code>	(none)	<code>1234567</code>	<code>12,345.67</code>
<code>MCU</code>	<code>10L</code>	<code>hello</code>	<code>HELLO</code>
<code>MCT</code>	(none)	<code>hello there world</code>	<code>Hello There World</code>
(none)	<code>10R</code>	<code>123</code>	<code>123</code>
<code>MX</code>	<code>8R</code>	<code>255</code>	<code>FF</code>

FIELD 3 (CONVERSION)	FIELD 5 (FORMAT)	STORED VALUE	OUTPUT
MB	(none)	5	101

Three of these are worth reading twice.

Internal date 20899 is 20 March 2025. Day zero is 31 December 1967, so an internal date does not correspond to anything you can guess.

D2/ produces 03/20/25 — month first. The bare **D** conversion produces **20 MAR 2025**, with the month abbreviated in capitals. If you want day first, say so in the conversion rather than relying on a default.

MD2 scales rather than truncating. **MD2** on 1234567 is **12345.67**, not **1234.56** — the digit count says where the decimal point goes, and no digits are lost.

I-type expressions — field 2 for type I

When the dictionary record is type **I**, field 2 is not a field number — it is an expression. The expression is compiled and executed for each record the query processor lists, and the result is the field value.

WHAT THE EXPRESSION CAN CONTAIN

An I-type expression is a single SD BASIC expression. It can use:

- **Field references:** **F2** is field 2 of the current record. **@RECORD** is the whole record as a dynamic array.
- **Pseudo-variables:** **@ID** (the record id), **@NI** (number of values in the longest multivalued field), **@NB** (record length), **@FILE.NAME**, **@USER.NO**, **@PATH**.
- **BASIC functions:** **LEN()**, **FIELD()**, **INDEX()**, **COUNT()**, **SUM()**, **OCONV()**, **ICONV()**, **DATE()**, **TIME()**, **RAISE()**, **DOWNCASE()**, **UPCASE()**, **TRIM()**, **STR()**, **SPACE()**, **SEQ()**, **CHAR()**, **ABS()**, **INT()**, **MOD()**, **SQRT()**, **TRANS()** and the rest.
- **Subroutine calls:** **SUBR("name", arg, ...)** invokes a catalogued subroutine. The **Ftype** I-type in **voc.dic** calls **SUBR("!Ftype", DATA.NAME)**.
- **Conditional expressions:** **IF condition THEN value ELSE value**. The **Ftype** I-type is **IF DATA.NAME # '' THEN SUBR("!Ftype", DATA.NAME) ELSE ''**.

WHAT THE EXPRESSION CANNOT CONTAIN

An I-type expression cannot use:

- **Statements:** no **OPEN**, **READ**, **WRITE**, **PRINT**, **INPUT**, **FOR**, **LOOP**, **GOTO**, **GOSUB**.
- **Multiple statements:** no semicolons between statements. The whole field 2 is one expression.
- **Dimensioned arrays:** no **DIM**, no **MAT**.
- **Common blocks:** no **COMMON**.

If you need any of these, write a subroutine, catalogue it, and call it with `SUBR()` from the I-type.

COMPILATION AND THE STAMP

The expression is compiled to p-code the first time it is used after it is written or edited. The object is stored in field 16 onward, and a hash of the source is stored in field 15. When a query opens the dictionary, it compares the hash to the current source — if they differ, the I-type is recompiled.

Editing the expression invalidates the object. You do not need to do anything to trigger recompilation — the next query that reads the field does it. If the expression has a syntax error, the query reports it and the field appears empty.

I-TYPE AND CONVERSION

An I-type record can have both an expression (field 2) and a conversion code (field 3). The expression runs first, producing a value, and the conversion is then applied to that value. An I-type that returns an internal date with `D` in field 3 will display it formatted — the expression computes the date, the conversion formats it.

Correlatives — field 8 for type A

A-type records carry a correlative in field 8. A correlative is a Pick-style expression — a short code that transforms the field value before display. SD compiles correlatives the same way it compiles I-types, storing the object in field 16 onward.

Correlative codes are a Pick inheritance and are rarely used in new SD dictionaries. An I-type expression is easier to read and can express calculations a correlative string cannot. The correlative mechanism exists so that dictionaries imported from Pick systems continue to work.

Chaining conversions

A conversion code can contain multiple conversions chained with value marks (char 253). Each is applied in turn to the result of the previous one. This is how a dictionary field can extract a substring, translate it, and format it in one pass — the value mark separates the steps.

Input and output conversion

The same conversion code works for both `OCONV()` (output) and `ICONV()` (input), but not every code is meaningful in both directions. `MCL` converts to lower case on output; on input it does the same. `D2[DD/MM/YY]` formats a date on output; on input it parses one. `MX` converts to hex on output; on input it converts from hex.

Some codes are output-only or input-only. `B64` encodes on output and decodes on input. `P(xx)` is a pattern test — it returns the original value or null on output, and accepts or rejects on input.

See also

[SD Dictionaries - Structure](#) · [SD VOC - Structure and Usage](#) · [SD TCL - The Query Processor](#) · [SD Basic - String Functions](#) · [SD Basic - Data Conversion](#).